

Memory Copy 用户手册

EIC7x 系列 AI 数字 SoC

Engineering Draft / Rev 0.4

2025-07-17

声明

版权所有©2024，北京奕斯伟计算技术有限公司及其关联公司（以下简称“ESWIN”）。保留所有权利。

ESWIN 在此软件中保留所有知识产权和专有权利。未经 ESWIN 授权，不得发布、复制、分发、转载、修改、改编、翻译或制作此软件的衍生作品，无论是全部还是部分。

对于本文档的任何修改，ESWIN 无需承担通知义务且不代表 ESWIN 做出任何承诺。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证与承诺，无论明示或默示。（本段落用于文档中）

“ESWIN” logo 和其他 ESWIN 标识，为北京奕斯伟计算技术有限公司及（或）其母公司所有的商标。本文档提及的其他所有商标或商品名称，由其各自的所有权人所有。

产品安装中可能包含开源软件和/或自由软件的许可声明。

修改历史

版本	日期	描述
V0.1	2024-11-04	初版。
V0.2	2024-12-03	修改了 cmdq 的实现方法。
V0.3	2024-12-18	API 优化。
V0.4	2025-07-17	API 优化。

目录

MEMORY COPY 用户手册	0
EIC7x 系列 AI 数字 SoC	0
目录	III
表目录	IV
1. 概述	1
2. API 简介	2
3. API 定义	3
4. 数据类型	7
5. ERRCODE	8
6. 附录	9

表目录

表 5-1 Error code.....	8
-----------------------	---

1. 概述

MemCP (Memory Copy) 模块提供了面向系统和应用程序的统一内存复制与命令队列管理接口。通过该模块，用户可以实现高效的异步数据传输和任务调度，支持复杂内存操作在多任务并发环境中的高效执行。MemCP 模块内核实现了命令队列 (CMDQ) 与内存复制 (Memcpy) 功能，提供了一致性强、易于使用的用户层接口。

CMDQ 功能在内核层通过 工作队列 (workqueue) 实现，专注于高效的异步任务调度和命令队列管理。其核心设计特点是支持多任务并发执行，同时确保任务按顺序或优先级正确执行，并能动态适应复杂的任务调度需求。通过 open 系统调用，系统为每个命令队列分配一个唯一的文件描述符 (fd)，该 fd 与内核中创建的工作队列绑定。每个 fd 对应一个独立的工作队列，支持多次创建，从而实现多个命令队列的独立管理和任务调度。CMDQ 允许用户同时为多个外设创建 DMA 任务队列，并利用内核调度机制动态分配 DMA 通道，确保数据搬运任务的高效完成。用户通过简单的接口调用即可完成命令队列的创建、任务添加、同步等待及状态查询，极大地降低了操作复杂性。同时，每个队列的资源在销毁时自动释放，确保系统资源的高效利用和管理。CMDQ 的这种设计，使其成为多设备、高并发任务场景下不可或缺的基础模块，为系统提供了高效的任务管理能力和灵活的扩展性。

MemCP 模块负责统一管理内存复制操作与命令队列 (CMDQ) 的创建、调度、同步等功能，提供高效、易用的内核接口支持异步任务处理和资源管理。通过 MemCP 模块，用户可以快速执行内存复制任务，并灵活管理命令队列，以满足复杂系统的并发操作需求。

通过这两个功能组合，系统能够在内核和用户空间之间、不同硬件设备之间，灵活地管理数据传输、命令执行和资源调度，从而保证系统的高效性和并发性。

MemCP 的主要功能包括：

- 命令队列创建与销毁：通过 open 和 close 系统调用创建和销毁命令队列。每个队列与一个文件描述符绑定，确保了多队列的独立性和资源的有效管理。
- 内核级异步任务调度：通过内核的工作队列 (workqueue) 机制实现异步任务的高效调度，用户只需将任务下发到命令队列中，内核会根据任务调度策略处理，用户无需等待任务完成即可执行其他操作。
- 内存复制与命令队列结合：支持直接通过命令队列下发内存复制任务，内存复制通过 DMA 通道实现，确保数据传输的高效性与可靠性。
- 命令同步与查询：支持对命令队列进行同步操作，确保所有命令执行完毕后才继续后续操作。同时，提供查询接口让用户获取队列的状态信息，包括当前任务执行进度、队列是否为空等。
- 错误处理与状态管理：提供详细的错误返回值，并记录日志信息。用户可以根据错误信息快速定位和解决问题。

系统用户态的 libes_memcp.so 库封装了 MemCP 模块的 API 接口，方便应用程序调用，从而实现高效的命令执行和内存管理。

2. API 简介

本章节介绍了 DMA 内存拷贝（Memory Copy，简称 MemCP）的相关 API。MemCP API 提供命令队列的创建、添加、异步内存拷贝、同步内存拷贝、查询和销毁等功能，以使用户管理队列中的命令执行和基于 DMA 的高效内存拷贝操作。

命令队列 API 提供了如下主要功能：

ES_CmdQueue_Create

创建并初始化一个命令队列，返回队列句柄。

【说明】

- 1、调用该接口将创建一个命令队列，并返回一个文件描述符 `cmdQHandle`，该文件描述符是命令队列的唯一标识。
- 2、创建的文件描述符与内核中的工作队列（workqueue）绑定，用于异步处理命令队列中的任务。
- 3、用户通过 `cmdQHandle` 对队列进行后续操作（如添加任务、同步、销毁等）。
- 4、每次调用 `ES_CmdQueue_Create`，都会创建一个新的命令队列和对应的工作队列。

ES_CmdQueue_Destroy

销毁命令队列，释放相关资源。

【说明】

- 1、通过传入的命令队列句柄 `cmdQHandle`，销毁与该句柄绑定的工作队列及其他资源。
- 2、调用此接口实际会调用系统的 `close` 函数，确保文件描述符关闭并释放其关联的内核资源。
- 3、在 `close` 操作完成后，队列及其任务将不再可用，相关资源将被回收。
- 4、确保在销毁队列前已完成所有任务或通过 `ES_CmdQueue_Sync` 确认任务完成，避免资源泄露或未完成任务的丢失。

ES_CmdQueue_Sync

阻塞调用线程，直到命令队列中的所有任务执行完成。

【说明】

- 1、通过命令队列句柄 `cmdQHandle` 确定需要同步的队列。
- 2、调用该接口后，调用线程将阻塞，直到队列中所有任务完成。
- 3、常用于确保所有任务在继续后续操作之前已处理完毕。

ES_CmdQueue_Query

查询命令队列的状态。

【说明】

- 1、返回命令队列当前状态，包括是否空闲（status 0 代表空闲 1 代表忙碌）以及剩余未完成的任务的数量（task_count）。
- 2、用户可通过该接口动态监控任务进度，用于实现任务调度的智能决策。

ES_Memcpy_Async

发起一个异步内存复制任务，并将任务添加到指定命令队列中。

【说明】

- 1、用户需提供源地址和目标地址的文件描述符（fd）、偏移量、复制长度和超时时间。
- 2、复制任务将在后台异步执行，不会阻塞调用线程。
- 3、内存复制操作通过 DMA 完成，系统自动分配 DMA 通道以实现高效传输。

【注意事项】

- 1、文件描述符（fd）是命令队列的核心标识符，任何操作都必须依赖于有效的 cmdQHandle。
- 2、严格遵循队列的创建与销毁流程，未及时销毁的队列可能会导致内核资源泄露。
- 3、每个队列 cmdQHandle 独立，支持多个命令队列并发使用。

ES_Memcpy_Async_Wait_Notifier

在异步内存拷贝任务提交后，等待并检测任务执行状态，专门用于捕获 DMA 操作超时（timeout）时内核返回的错误。

【说明】

- 1、监测有效的 cmdQHandle 描述符上的拷贝任务。
- 2、替代 ES_CmdQueue_Sync 同步操作；
- 2、当 ES_Memcpy_Async 设置的超时时间（timeout 参数）触发后，内核无法通过 ioctl 直接返回错误，通过等待内核事件通知获取实际错误，弥补 ioctl 异步提交的缺陷。

【注意事项】

- 1、必须紧接在 ES_Memcpy_Async 后调用，针对同一队列中的最近提交任务。
- 2、使用此接口不需要再 ES_CmdQueue_Sync；

3. API 定义**ES_CmdQueue_Create****【函数体】**

```
ES_S32 ES_CmdQueue_Create(ES_S32 *cmdQHandle)
```

【描述】

创建一个新的命令队列，用于管理异步操作。该队列通过文件描述符 `cmdQHandle` 进行标识和管理，并将与内核的工作队列（workqueue）进行绑定。

【参数】

Parameter Name	描述 s	Input/Output
<code>cmdQHandle</code>	返回的命令队列句柄，用于后续操作	Output

【返回值】

Return Value	描述 s
0	Success
None 0	Fail, refers to the <code>_ErrCode</code>

【注意】

无

ES_CmdQueue_Destroy

【函数体】

ES_S32 ES_CmdQueue_Destroy(ES_S32 cmdQHandle)

【描述】

销毁一个已创建的命令队列并释放所有相关资源。调用此接口将关闭与命令队列关联的文件描述符，确保资源正确回收。

【参数】

Parameter Name	描述 s	Input/Output
<code>cmdQHandle</code>	要销毁的命令队列句柄	Input

【返回值】

Return Value	描述 s
0	Success
None 0	Fail, refers to the <code>_ErrCode</code>

【注意】

在使用完命令队列后应调用 `ES_CmdQueue_Destroy` 销毁，以释放资源。

ES_CmdQueue_Sync

【函数体】

ES_S32 ES_CmdQueue_Sync(ES_S32 cmdQHandle)

【描述】

同步等待指定命令队列中所有任务执行完成，确保任务执行的顺序性和一致性。

【参数】

Parameter Name	描述 s	Input/Output
cmdQHandle	要同步的命令队列句柄	Input

【返回值】

Return Value	描述 s
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

无

ES_CmdQueue_Query

【函数体】

ES_S32 ES_CmdQueue_Query(ES_S32 cmdQHandle, struct esw_cmdq_query *query)

【描述】

查询指定命令队列的当前状态，包括队列任务的完成情况、任务数量等。

【参数】

Parameter Name	描述 s	Input/Output
cmdQHandle	要同步的命令队列句柄	Input
query	用于存储查询结构的结构体	Output

【返回值】

Return Value	描述 s
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

无

ES_Memcpy_Async

【函数体】

ES_S32 ES_Memcpy_Async(
 ES_S32 cmdHandle,
 ES_U64 srcFd,

ES_U32 srcOffset,
ES_U64 dstFd,
ES_U32 dstOffset,
ES_U64 len,
ES_S32 timeout)

【描述】

执行同步内存复制操作，等待复制完成后返回。

【参数】

Parameter Name	描述 s	Input/Output
cmdQHandle	目标命令队列的句柄，用于分配任务	Input
srcFd	源文件描述符，表示源内存的 DMA-BUF 句柄	Input
srcOffset	源内存偏移量，单位为字节	Input
dstFd	目标文件描述符，表示目标内存的 DMA-BUF 句柄	Input
dstOffset	目标内存偏移量，单位为字节	Input
len	要复制的数据长度，以字节为单位	Input
timeout	内存赋值操作超时时间，以毫秒为单位	Input

【返回值】

Return Value	描述 s
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

无

ES_Memcpy_Async_Wait_Notifier

【函数体】

ES_S32 ES_Memcpy_Async_Wait_Notifier(ES_S32 cmdQHandle)

【描述】

在调用 ES_Memcpy_Async 后检测异步内存拷贝任务的状态，用于捕获 DMA 传输超时错误。

【参数】

Parameter Name	描述 s	Input/Output
cmdQHandle	目标命令队列的句柄，用于分配任务	Input

【返回值】

Return Value	描述 s
0	Success
None 0	Fail, refers to the _ErrCode

【注意】

无

4. 数据类型

相关数据类型、数据结构定义如下：

- [esw_cmdq_query](#):定义命令队列的查询结果结构体，用于返回当前命令队列的状态信息。

esw_cmdq_query

【说明】

定义命令队列的查询结果结构体，用于返回当前命令队列的状态信息。

【定义】

```
struct esw_cmdq_query {
    int status;
    int task_count;
    int last_error;
};
```

【成员】

Member	描述 s
status	队列状态，0 表示空闲（FREE），1 表示繁忙（BUSY）。
task_count	队列中当前未完成任务的数量。
last_error	最后一个失败任务的错误码。

- [esw_memcp_f2f_cmd](#):定义从源缓冲区到目标缓冲区的内存拷贝命令结构。

esw_memcp_f2f_cmd

【说明】

定义了一个结构体，用于表示从源缓冲区到目标缓冲区的内存拷贝命令，包括源和目标的文件描述符、偏移量、数据长度以及操作超时时间。。

【定义】

```
struct esw_memcp_f2f_cmd {
    int src_fd;
    unsigned int src_offset;
```

```

int dst_fd;
unsigned int dst_offset;
size_t len;
int timeout;
};

```

【成员】

Member	描述 s
src_fd	源缓冲区的文件描述符。
src_offset	源缓冲区中数据拷贝开始的偏移量。
dst_fd	目标缓冲区的文件描述符。
dst_offset	目标缓冲区中数据将被拷贝到的偏移量。
len	要拷贝的数据长度，单位为字节
timeout	内存拷贝操作的超时时间，单位为毫秒。

5. ErrCode

表 5-1 Error code

ErroCode	Macro Definition	描述
0xA0156009	ES_MEMCP_ERROR_NULL_PTR	A null pointer was encountered where a valid pointer was expected.
0xA015600A	ES_MEMCP_BUSY	Failed to process request because the command queue is busy.
0xA015600B	ES_MEMCP_INVALID_HANDLE	Invalid command queue handle specified.
0xA015600C	ES_MEMCP_MEMORY_ERROR	Memory allocation failed for command queue operations.
0xA015600D	ES_MEMCP_OPERATION_FAIL	Command queue operation failed due to an unknown error.
0xA015600E	ES_MEMCP_UNKNOWN_CMD	Command queue received an unknown or unsupported command.
0xA015600F	ES_MEMCP_SYNC_ERROR	Synchronization error occurred within the command queue.
0xA0156010	ES_MEMCP_DEV_NOT_FOUND	Memory copy device not found.
0xA0156011	ES_MEMCP_IOCTL_FAIL	IOCTL operation failed for memory copy device.
0xA0156012	ES_MEMCP_MEM_ALLOC_FAIL	Memory allocation failed during memory copy operation.
0xA0156013	ES_MEMCP_DMA_TIMEOUT	DMA transfer timeout error.

6. 附录

使用说明

- 1、使用前确保板端运行环境下交付件中所有驱动 ko 文件已正常加载
- 2、在板端运行环境下/usr/lib 下存在动态库 libes_memcp.so 和 libes_memory.so